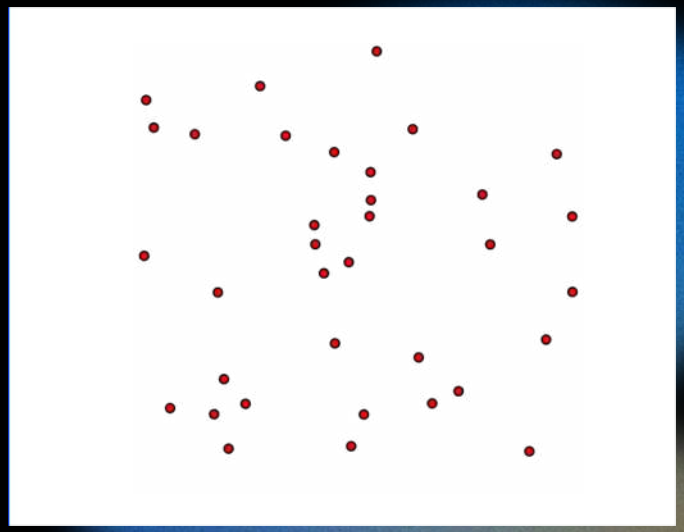
# TSP Problem: From Mathematics to Art

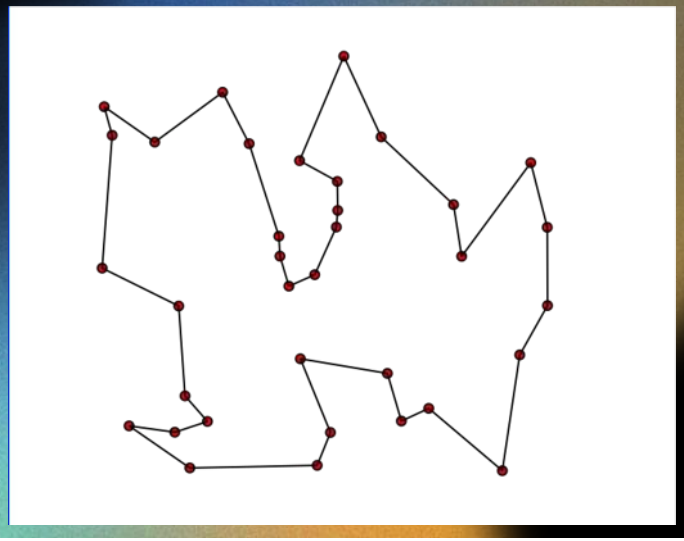
By: Dengyuhan Dai

# What's TSP?

- *Def.* "Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?"

- Can be modeled as a graph problem: undirected weighted graph





# Pipeline

## Step 1: Preprocessing
Sample photo to a density field

## Step 2: Generate cities
higher density -> more cities

## Step 3: TSP Solver
Use packaged solver to get the solution

## Step 4: Drawing
Use the solution to TSP problem to draw the desired result picture

# 1.Preprocessing

Intuition: a place either exists a city, either not.

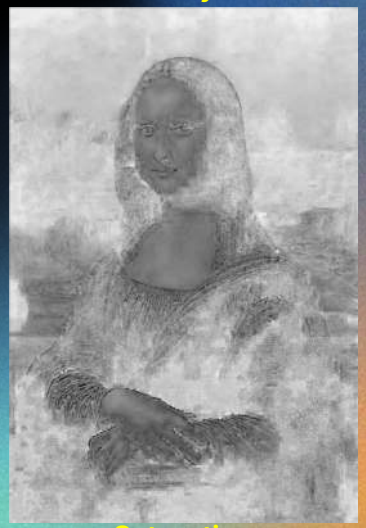
Thus original sample -> binary image;
View such image as density field & extract cities.

Several ways to do this:

- Gray scale
- Edge(intense changes in  RGB -> darker)
- Combining of Gray and Edge
- Saturation

Gray

Edge



Saturation

Gray+Edge

# 2.Generate cities

If a pixel is not white, then pick it as a city.

- Use an upper bound max_cities to control the overall amounts of cities.
- If passed, drop some cities randomly.

```python
def extract_cities_from_dithered(
    border_margin_px: int = 0,
) -> np.ndarray:
    """
    Extract city coordinates (x,y) from a binary dithered image where black pixels are cities.

    border_margin_px:
        If > 0, drop any cities within `border_margin_px` of the image boundary.
        This is a strict, cheap way to eliminate edge outliers that often cause very long TSP edges.
    """
    h, w = dithered_u8.shape[:2]
    ys, xs = np.where(dithered_u8 == 0)

    if len(xs) == 0:
        return np.zeros((0, 2), dtype=int)

    m = int(border_margin_px)
    if m > 0:
        keep = (xs >= m) & (xs < (w - m)) & (ys >= m) & (ys < (h - m))
        xs = xs[keep]
        ys = ys[keep]
        if len(xs) == 0:
            return np.zeros((0, 2), dtype=int)

    cities = np.stack([xs, ys], axis=1).astype(int)

    if len(cities) > max_cities:
        rng = np.random.default_rng(seed)
        idx = rng.choice(len(cities), max_cities, replace=False)
        cities = cities[idx]

    return cities
```

# 3.Solver(LKH-3)

- Generate a simple initial solution(greedy)
- K-opt tree: continuously replace k edges; if the overall length is shortened, try k+1.
- repeat

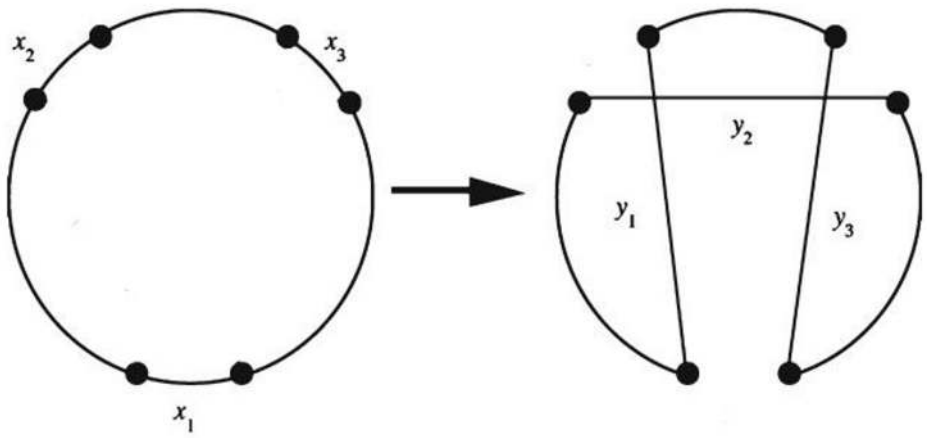
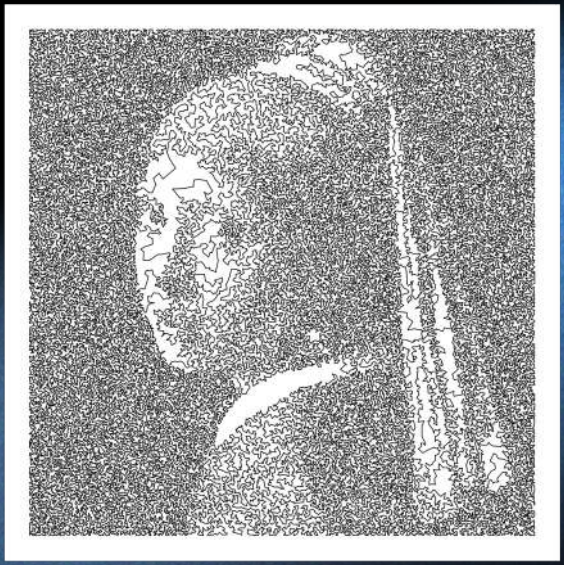
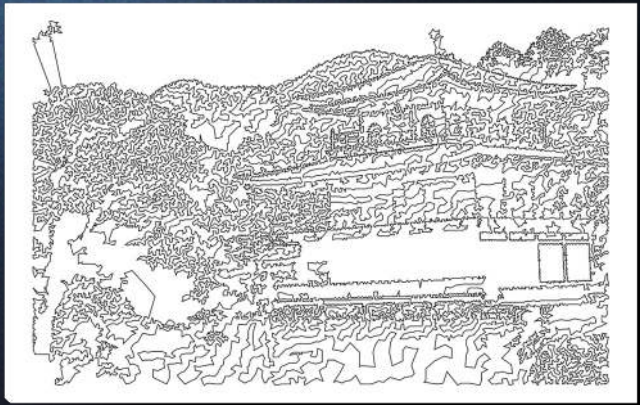
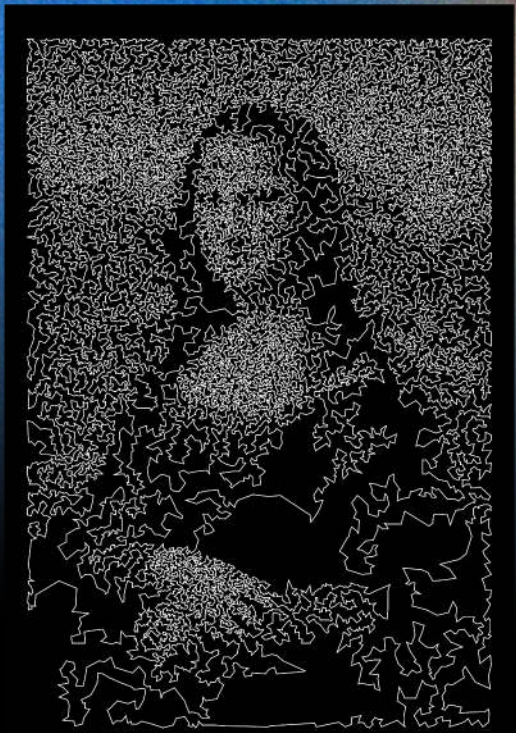



**Fig. 1** A 3-opt move. $x_1, x_2, x_3$ are replaced by $y_1, y_2, y_3$

Source: K. Helsgaun, "General k-opt submoves for the Lin–Kernighan TSP heuristic,"
Mathematical Programming Computation, 2009.



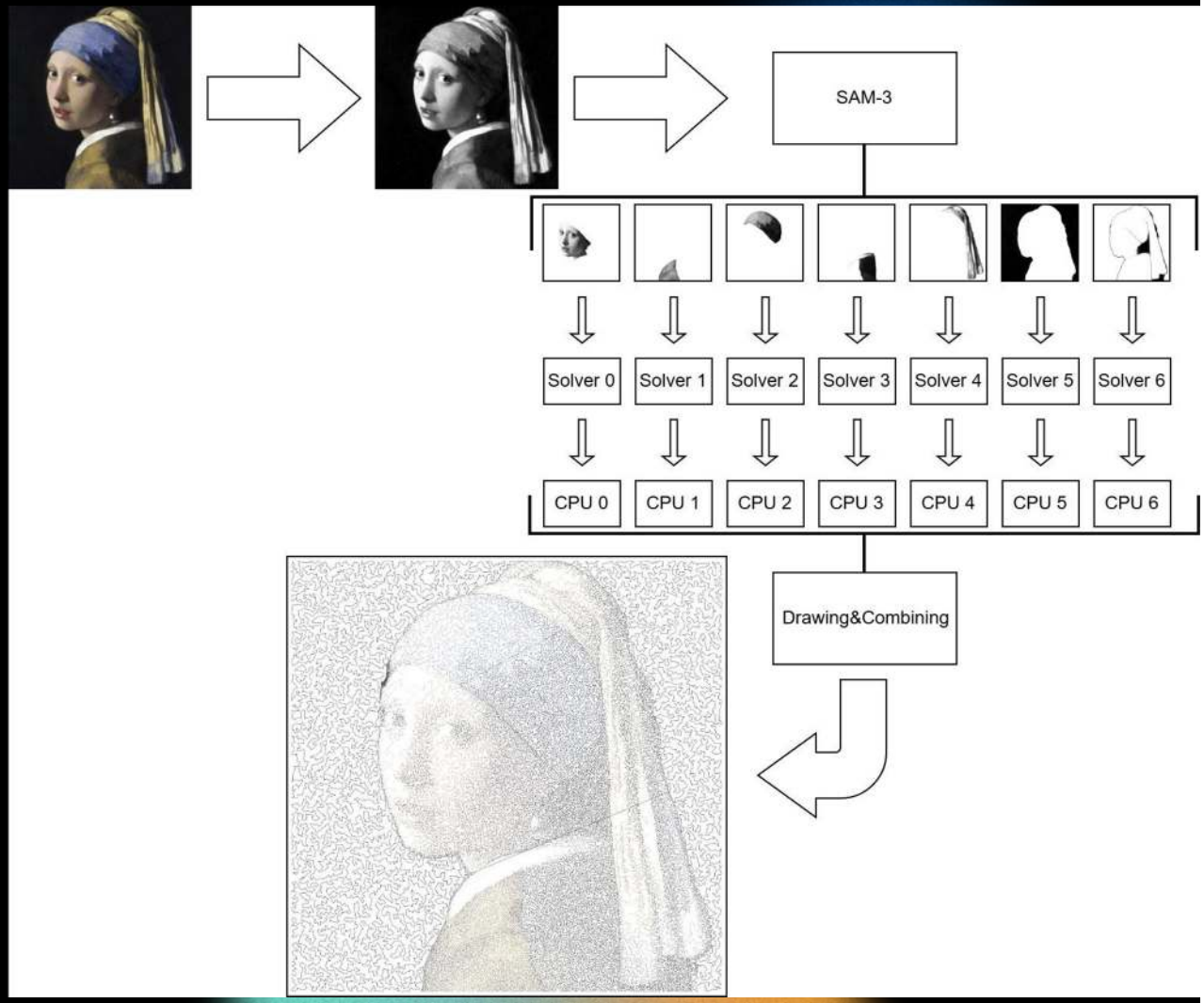




# 4.Drawing

# SAM-3&Parallelization

- **“Divide&Conquer” by SAM-3 segmentation**
- Parallelism for solver processes: initiate multiple solver processes and run seperately on CPU cores



# Colorable drawing

- Colorable drawing method: for each edges, find the middle points of two cities on the end, and then take the RGB value from original sample

- **Stats:**

  **Cities: 229, 362**

  **Layers: 6**

  **Preprocessing method: gray**

# Thank you